# Reporte Técnico de Actividades Práctico-Experimentales 01

## 1. Datos de Identificación del Estudiante y la Práctica

| | |
|---|---|
| **Nombre de los estudiante** | Tayron Fernando Morales Villamagua<br>Ariana Alejandra Córdova Betancourt<br>Jorge Luis Luzuriaga Betancourt<br>Emilio Ramiro Jaramillo |
| **Asignatura** | Sistemas Distribuidos |
| **Ciclo** | 6to |
| **Unidad** | Envío de mensajes distribuido |
| **Resultado de aprendizaje de la unidad** | R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 01 |
| **Título de la Práctica** | Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido |
| **Nombre del Docente** | José O. Guamán Q |
| **Fecha** | 29 de septiembre de 2026<br>02 de octubre de 2026 |
| **Horario** | Martes 10h30 a 11h30<br>Viernes 07h30 a 09h30 |
| **Lugar** | Aula |
| **Tiempo planificado en el Sílabo** | 3 |

## 2. Objetivo(s) de la Práctica

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuada, justificando técnicamente la elección del equipo

## 3. Materiales y Reactivos

Diapositivas, y enunciados del problema:

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

## 4. Equipos y Herramientas

Computadora personal con conexión a internet

Guía de práctica.

Plantilla de matriz comparativa

## 5. Procedimiento / Metodología Ejecutada

Fase 0 (10 min). Formación de equipos de 4 (asignación del docente).

Fase 1 – Investigación (50 min). Cada integrante investiga su arquitectura y completa su sección de la matriz (estructura, ventajas, desventajas, escalabilidad, ejemplos). El equipo elabora el diagrama de su arquitectura en draw.io.

Fase 2 – Socialización (30 min). Cada equipo expone 5 min. El resto completa la matriz comparativa con la información expuesta (una sola matriz, grupal).

Fase 3 – Caso de estudio (55 min). Cada equipo selecciona la arquitectura más adecuada para el enunciado, la justifica con mínimo 3 argumentos técnicos, elabora el diagrama de la solución e identifica dónde se aplican transparencia, escalabilidad y tolerancia a fallos.

Fase 4 – Reporte y entrega (25 min). Cada estudiante redacta sus conclusiones individuales; el equipo compila la matriz final y los diagramas. Entrega en la plataforma indicada.

**FASE 1**
Diagramas de arquitectura**:**

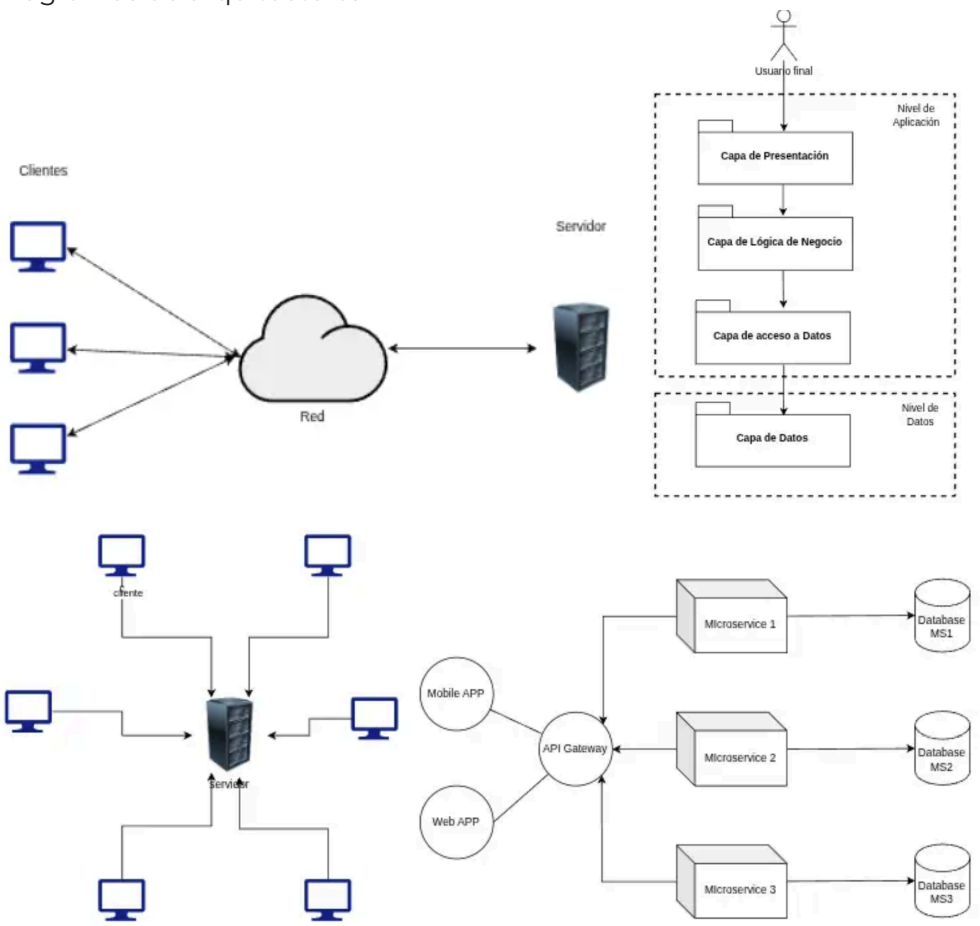



**Figura 1: [Diagrama Drawn.io]**

Matriz de investigación

| Criterio | Cliente-Servidor | Peer-to-Peer | Multicapa | Microservicios |
|---|---|---|---|---|
| **Estructura** | La arquitectura es:<br>-Cliente: dispositivo o aplicación que interactúa directamente con el usuario.<br>-Servidor: espera las peticiones de los clientes, y devuelve una respuesta.<br>-Red: medio físico o inalámbrico. | Se la puede considerar una red virtual superpuesta sobre la infraestructura física de una red las cuales son:<br>-Simetría de Roles.<br>-Ubicación en los bordes de la red.<br>-Y modelos topológicos de la red: | La arquitectura se divide en:<br>- Capa de Presentación<br>- Capa de lógica de negocio.<br>- Capa de acceso a datos<br>- Capa de datos<br>Cabe mencionar que esta arquitectura no se limita a tener tres o cuatro capas, ya que | La arquitectura está compuesta por:<br>- Puerta de Enlace (API Gateway)<br>- Microservicios Independientes<br>- Bases de Datos por Servicio<br>- Puerto de Servicios |

| | | | | |
|---|---|---|---|---|
| | | Semicentralizado , Descentralizado no estructurado y Descentralizado estructurado | podría aplicar más capas dependiendo de las necesidades. | |
| **Ventajas** | -Centraliza la información y los recursos.<br>-Permite que varios clientes utilicen los mismos servicios.<br>-El mantenimiento se realiza directamente en el servidor sin necesidad de modificar cada cliente. | -Tiene colaboración directa a intercambio de recursos.<br>-Reducción de costos de infraestructura.<br>-Alta disponibilidad y tolerancia a fallos. | Gracias a la separación por capas se nos permite:<br>- Administrar cada capa de forma individual.<br>- Un buen mantenimiento y actualización de aplicaciones de gran escala.<br>- Manejar y probar capas de manera independiente. | ● Autonomía del Equipo<br>● Servicios Simples<br>● Canalización de implementación rápida<br>● Múltiples pilas de tecnología<br>● Subdominios caracterizados |
| **Desventajas** | -Si el servidor falla, los clientes pueden quedarse sin servicio.<br>-Un servidor con demasiadas solicitudes puede saturarse. | -Posee dificultades de direccionamiento<br>-Volatilidad de la red<br>-Consumo masivo de banda Ancha<br>-Riesgo de seguridad y confianza | - Al aplicar varias capas una operación sencilla puede pasar por varias capas innecesariamente.<br>- Las peticiones pasan por varias capas agregando cierta sobrecarga<br>- En proyectos pequeños puede ser innecesario. | ● Interacciones complejas o ineficientes<br>● Complejidad Operativa Alta<br>● Gestión de Datos Fragmentados<br>● Pruebas y Depuraciones complicadas |
| **Escabilidad** | Pueden haber dos formas:<br>-Escalabilidad vertical: Aumentar los recursos de un servidor.<br>-Escalabilidad horizontal: Agregar más servidores. | Posee dos tipos de escalabilidad:<br>Escalabilidad lineal: más usuarios más demanda de recursos<br>Escalabilidad de redes masivas: limita la información permitiendo que el sistema escale | Posee escalabilidad vertical: Permite agregar los recursos de un servidor, como memoria RAM, CPU.<br>Y escalabilidad horizontal, la cual agrega más servidores. | Escalabilidad Horizontal: Se pueden crear más instancias de servidores y conectarlos.<br>Escalabilidad Vertical: Se puede mejorar los recursos de un solo servidor |
| **Ejemplos Reales** | Un teléfono o computadora ejecuta la aplicación o el navegador (el cliente). Cuando revisas tu saldo o realizas una transferencia, la petición viaja a los | Compartición de archivos y almacenamiento distribuido:<br>-Napster: Usaba un directorio central para localizar archivos. | Un sistema de facturación de cuatro capas:<br>- Presentación: Interfaz donde el usuario ingresa y modifica datos de clientes.<br>- Negocio: Procesa las | Amazon: En su plataforma de comercio electrónico, el servicio de Inventario tiene su propia base de datos y el servicio de Pedidos tiene otra diferente. Si el servicio de pedidos |

| | | | | |
|---|---|---|---|---|
| | servidores centrales del banco (el servidor), los cuales validan tu identidad, consultan la base de datos de cuentas y devuelven el resultado a la pantalla. | -Gnutella: Funcionaba de forma descentralizada, buscando archivos entre los usuarios.<br>-KaZaA / FastTrack: Utilizaban supernodos para organizar y mejorar la búsqueda de archivos. | solicitudes de la interfaz<br>- Entidad: Define los datos del cliente permitiendo transportarlos por las diferentes capas<br>- Datos: Se comunica con la base de datos. | necesita saber el stock, debe solicitarlo mediante una API al servicio de inventario, evitando el acoplamiento directo a nivel de datos.<br>Uber: Cuando pides un viaje, intervienen el Servicio de Viajes, el Servicio de Conductores y el Servicio de Cobros. Si el cobro con tarjeta falla después de asignar al conductor, el patrón Saga ejecuta una transacción compensatoria para cancelar la asignación del conductor de forma automática. |

**FASE 2**

Matriz comparativa con la información expuesta

| Grupo | Criterio | Cliente Servidor | Peer-to-Peer | Multicapa | Microservicios |
|---|---|---|---|---|---|
| **Grupo 1** | **Estructura** | Conformada por dos nodos cliente y servidor, el nodo cliente se encarga de interactiva con el usuario y mientras tanto el nodo servidor se encarga de administrar la información de los clientes | Modelo de comunicación en red donde no existen servidores ni clientes fijos, sino que todos los nodos (computadores) se comportan como iguales entre sí. | Se conforma por varias capas de las cuales se distribuyen diferentes responsabilidades. Las capas identificadas más regulares son: Capa de presentación, Capa de acceso y capa de datos. | Se divide en servicios pequeños que trabajan juntos, cada microservicio se encarga de una función concreta del negocio y puede ejecutarse de manera independiente |
| | **Ventajas** | -Permite que varios clientes utilicen los mismos servicios.<br>-El mantenimiento se realiza | -Escalabilidad alta, ilimitada a nivel global.<br>-Cada nodo puede compartir información. | Mejor organización jerárquica.<br>Responsabilidad por cada capa<br>No existe el riesgo de que una capa | Modificar, corregir y desplegar un servicio sin tener que desplegar nuevamente toda la aplicación, |

| | | | | | |
|---|---|---|---|---|---|
| | | directamente en el servidor sin necesidad de modificar cada cliente. | | interfiera con otra capa y dañe la estructura. | distintos equipos se encarguen de servicios específicos y trabajen con mayor autonomía |
| | **Desventajas** | Si hay muchos clientes se satura el servidor | -Alta volatilidad de nodos.<br><br>-Información repetida entre nodos. | El sistema tiene muchos nodos o servicios que dificulta un proceso que tiene que atravesar muchas capas y provoca latencia | Demora en la llamada a cada servicio que causa una latencia en el sistema y llega a caerse |
| | **Escabilidad** | Se puede agregar más recursos al servidor para manejar más solicitudes. | Escalailidad alta y global. | Se puede aumentar recursos cuando aumenta el numero de solicitud. | Si una función tiene más demanda que las demás, se añaden más instancias del servicio encargado de atenderla |
| | **Ejemplos reales** | Un sistema Universitario ya que el estudiante | Comunicación: Skype y Torrent | Sistema bancario que maneja una base de datos sensibles por lo que se divide en capa de presentacion, de logica de negocio | Netflix Y Sound Cloud |

| | | | | | |
|---|---|---|---|---|---|
| **Grupo 2** | **Estructura** | Dos componentes:<br>-Cliente<br>-Servidor<br>-Protocolos | Arquitectura que consta que hay nodos que se conectan entre sí.<br>-Estructurada<br>-No estructurada<br>-Mixto | Organiza una aplicación en varias capas horizontales, donde cada una tiene un rol específico siguiendo el principio de separación de responsabilidad | Desglose de componentes independientes que se pueden desplegarse en distintos servicios, comunicados a través de API. Servicio, Gateway, Cliente |
| | **Ventajas** | Servidor centralizado al | Si uno falla todo sigue | - La separación de | Libertad de Elección |

| | | | | | |
|---|---|---|---|---|---|
| | | que todos los clientes pueden ingresar. | funcionando Permite compartir información de mejor manera Optimización de red | responsabilidades permite que cada capa sea independiente.<br>- Un error no se propague a todas las capas.<br>- Se pueden hacer pruebas o simulaciones en capas adyacentes | Tecnológica, facilidad de mantenimiento, gran resiliencia si un componente falla. |
| | **Desventajas** | En caso de que el servidor caiga, todos los servicios caerán. | Fallo en nodo si falla uno es difícil encontrarlo No todos los nodos tienen la misma ventaja y sincronización de datos | Cuando muchas peticiones atraviesan afectan el rendimiento y produce latencia | Alta complejidad de Gestión, coherencia o consistencia, costes de mantenimiento. |
| | **Escalabilidad** | Entre más clientes, se puede aumentar la capacidad del servidor | Permite que haya muchos datos y nodos | Supera lo que es un monolito pero no llega tanto a lo de microservicios, su fortaleza está en la división de responsabilidades en cada capa | Aporta una escalabilidad |
| | **Ejemplos reales** | -Netflix<br>-Youtube | -Bitcoin<br>Y bancos | ConStore y libreria Java Health Watcher Aplication, cada uno de estos se divide en capas independientes acordes a las necesidades de cada necesidades | Spotify y Servidor financiero como J.P. Morgan |

| | | | | | |
|---|---|---|---|---|---|
| **Grupo 4** | **Estructura** | Cliente realiza solicitudes y el | Es una red de dispositivos | se organiza en tres niveles | Enrutamiento segmentado: |

| | | | | | |
|---|---|---|---|---|---|
| | | servidor las procesa. | cliente servidor Centralizado Híbridos | fundamentales Capa de presentación, corresponde a la interfaz de usuario /presentación Capa media: Lógica de negocio / servidor de aplicaciones Capa de almacenamiento: Base de datos /servidor | Frontend, API Gateway, Microservicios independientes, Base de datos |
| | **Ventajas** | Centralización de la información. | Capacidad de compartir Archivos entre varios usuarios | Los datos frecuentes pueden almacenarse en cachés ubicados en la capa media lo que permite que la información se obtenga de manera más rápida. | Independencia total (actualizar, testear y desplegar sin afectar al resto).<br><br>Libertad de lenguajes propios por módulo. |
| | **Desventajas** | Muchos clientes pueden colapsar el servidor. | Optimización y Carga a la banda ancha | Cuando muchas peticiones atraviesan afectan el rendimiento y produce latencia | Mayor riesgo de filtración de datos (aumentan puntos de inseguridad).<br>Alto consumo sumado de recursos.<br>Red de comunicación compleja que genera probl de latencia |
| | **Escalabilidad** | Se puede agregar más recursos al servidor para manejar mas solicitudes. | Se considera infinito ya que depende cuantas pcs esten | Distribución de las carga entre múltiples capas, lo que facilita el crecimiento del sistema cuando aumenta el número de | Alta y Granular (Inversión por servicio específico) |

| | | | | | |
|---|---|---|---|---|---|
| | | | | solicitud | |
| | **Ejemplos reales** | Un profesor puede tener demasiados estudiantes, y si tienen demasiadas dudas pueden aturdir al profesor. | -Bitcoins<br>No depende de en entidad externas | Sistema Universitario: gestión de matriculas cumple con las tres capas principales, presentación, lógica de negocio y la capa de datos. | Billeteras digitales (Deuna), Apps de Food |

| | | | | | |
|---|---|---|---|---|---|
| **Grupo 5** | **Estructura** | Cliente que realiza peticiones, y servidor que las responde. | Modelo de comunicación en red donde no existen servidores ni clientes fijos, sino que todos los nodos (computadores) se comportan como iguales entre sí. | Se conforma normalmente por: Capa de presentación, capa de lógica de negocio y capa de datos | Enrutamiento segmentado: Frontend, API Gateway, Microservicios independientes, Base de datos |
| | **Ventajas** | Arquitectura centralizada, en la que todos los usuarios pueden acceder a la vez. | -Escalabilidad alta, ilimitada a nivel global.<br>-Cada nodo puede compartir información. | Más utilizada y versátil<br>Fácilmente escalable | Independencia total (actualizar, testear y desplegar sin afectar al resto).<br><br>Libertad de lenguajes propios por módulo. |
| | **Desventajas** | Si el servidor falla o hay gran cantidad de usuarios puede haber retraso. | -Alta volatilidad de nodos.<br><br>-Información repetida entre nodos. | Pérdida de rendimiento e incremento de la latencia.<br>Se requiere escribir más código repetitivo y el diseño inicial se vuelve más complejo de planificar. | Mayor riesgo de filtración de datos (aumentan puntos de inseguridad).<br>Alto consumo sumado de recursos.<br>Red de comunicación compleja que genera probl de latencia |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | |
| | **Escalabilidad** | El servidor puede ser mejorado en cuanto a hardware. | Escalabilidad alta y global. | Permite que cada capa escale de manera independiente según requiera el sistema. | Alta y Granular (Inversión por servicio específico) |
| | **Ejemplos reales** | Juego en línea, la computadora es el cliente que recibe los inputs que realiza el usuario, se envían los inputs mediante el canal que es el internet, el servidor procesa los inputs de los clientes, los procesa y devuelve la respuesta. | Windows Delivery Optimization Hace que mi computadora busque un dispositivo cercano para poder buscar archivos faltantes | Sistema de banca en línea<br>- Capa de presentación<br>- Capa de negocio<br>- Capa de | Uber y Spotify |

| | | | | | |
|---|---|---|---|---|---|
| **Grupo 6** | **Estructura** | Cliente que realiza peticiones, y servidor que las responde y se comunican mediante HTTP. | Modelo de comunicación en red donde no existen servidores ni clientes fijos, sino que todos los nodos (computadores) se comportan como iguales entre sí. | | Servicios pequeños independientes, comunicados via red. |
| | **Ventajas** | Un único dispositivo por que se puede trabajar | -Escalabilidad alta, ilimitada a nivel global.<br>-Cada nodo puede compartir | | despliegue independiente, tolerancia a fallos,. |

| | | | | | |
|---|---|---|---|---|---|
| | | | información. | | |
| | **Desventajas** | Cuello de botella, si se daña se daña todo el servidor. | -Alta volatilidad de nodos.<br><br>-Información repetida entre nodos. | | latencia pro viajes de re, consistencia de datos compleja. |
| | **Escalabilidad** | El servidor puede ser mejorado en cuanto a hardware. | Escalailidad alta y global. | | horizontal automatizada, stateless, orquestadores automatizan el escalado de instancias |
| | **Ejemplos reales** | Servicio de Gmail, donde se envían correos y se nos devuelven. | Base de Datos Gnullet | | |

| | | | | | |
|---|---|---|---|---|---|
| **Grupo 7** | **Estructura** | Cliente que realiza solicitudes, y el servidor le provee lo que solicita. | Modelo de comunicación en red donde no existen servidores ni clientes fijos, sino que todos los nodos (computadores) se comportan como iguales entre sí. | | Enrutamiento segmentado: Frontend, API Gateway, Microservicios independientes, Base de datos |
| | **Ventajas** | Seguridad y manejo de usuarios. | -Escalabilidad alta, ilimitada a nivel global.<br><br>-Cada nodo puede compartir información. | | Independencia total (actualizar, testear y desplegar sin afectar al resto).<br><br>Libertad de lenguajes propios por módulo. |
| | **Desventajas** | Demasiadas solicitudes | -Alta volatilidad de nodos. | | Mayor riesgo de filtración de |

| | | | | | |
|---|---|---|---|---|---|
| | | pueden presentar fallos en la latencia. | -Información repetida entre nodos. | | datos (aumentan puntos de inseguridad). Alto consumo sumado de recursos. Red de comunicación compleja que genera probl de latencia |
| | **Escalabilidad** | El servidor puede ser mejorado en cuanto a hardware. | Escalailidad alta y global. | | Alta y Granular (Inversión por servicio específico) |
| | **Ejemplos reales** | Banco de Loja: el usuario envía la solicitud al servidor bancario, se procesa y se valida, se consulta a la base de datos, se confirma y se devuelve la respuesta al usuario. | Comunicación: Skype y Torrent | Tienda universitaria que se dividió en copa de presentación, capa de aplicación, capa de base de datos. | |

**FASE 3**

Selección de la Arquitectura

Para garantizar la continuidad operativa de la recepción de materia prima en un ingenio azucarero (proceso de misión crítica expuesto a cortes de comunicación), se selecciona la Arquitectura Multicapa con patrón Servidor-Cliente local, habilitada para funcionar en Modo Sincrónico y Asincrónico

Justificación Técnica

Aislamiento de fallas en la red WAN: La arquitectura introduce un nodo Servidor-Cliente local que actúa como servidor ante las estaciones de pesaje y como cliente frente al Data Center central

Esto independiza la operación en planta de la disponibilidad del enlace remoto

Persistencia y resincronización diferida de datos: Al interrumpirse la conexión WAN, la planta entra en Modo Asincrónico, registrando las transacciones en una base de datos local Una vez restablecida la red, servicios en segundo plano sincronizan automáticamente la información con el sistema central mediante reglas de resolución de conflictos

Alta relación costo/beneficio y reuso de infraestructura: Permite mantener el abastecimiento ininterrumpido aprovechando la red LAN local existente y

evitando pérdidas por paradas de fábrica, sin requerir reestructurar completamente los sistemas corporativos

**FASE 4**
**Conclusión Tayron Morales**

La elección entre esquemas cliente-servidor, peer-to-peer, multicapa y microservicios depende de equilibrar el acoplamiento de componentes, los costos de infraestructura y las necesidades operativas del negocio, pues ningún modelo es universalmente superior

La fortaleza de un sistema distribuido reside en ocultar su complejidad al usuario final (transparencia) y garantizar la continuidad del servicio mediante replicación, operación asincrónica y tolerancia a fallos parciales

**Conclusión Jorge Luzuriaga**

Cada ingeniero tiene distintas formas de afrontar un problema, cuando se le pide elegir la arquitectura necesaria para lo que pide el usuario, puede elegir alguna de las que se ha expuesto en este trabajo. Es trabajo del ingeniero conocer cada posibilidad que tiene y elegir de acuerdo al contexto que se le presente, ya sea en base al presupuesto, alcance del proyecto o gusto del usuario final.

**Conclusión Ariana Córdova**

Actualmente tenemos variedad de opciones de arquitecturas (Per-to-per, Cliente-servidor, multicapas, microservicios) que podemos aplicar dentro de nuestro sistema, y como estudiantes y futuros profesionales es nuestro deber investigar y aprender de cada una de las tecnologías existentes para poder tomar las decisiones correctas al momento de aplicar una arquitectura dentro de nuestro proyecto, es por ello que considero que esta actividad fue fundamental para nuestra preparación personal.

**Conclusión Emilio Jaramillo**

Existen muchas arquitecturas principales de un sistema distribuido, como las vistas en este trabajo, cada una presenta su diferente estructura, tienen diferentes ventajas dependiendo cada una de sus funciones, y todas presentan ciertas desventajas, presentes en varios ejemplos funcionales, y cada una tiene una escalabilidad diferente, esto quiere decir que será el diseñador quien seleccione la arquitectura a utilizar cual de estos parámetros sera el mas efectivo para desarrollar y tener un sistema funcional.

## 6. Preguntas de Control

**¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?**

Un sistema distribuido está formado por varios computadores que trabajan juntos y se comunican mediante una red para realizar tareas. Se diferencia de un sistema centralizado porque no depende de un único equipo. Sus características principales son: compartición de recursos, comunicación entre nodos, concurrencia, escalabilidad, tolerancia a fallos y transparencia.

**Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.**

En la arquitectura cliente-servidor, los clientes solicitan servicios y un servidor centralizado responde a esas solicitudes. En peer-to-peer (P2P), los dispositivos

pueden actuar como clientes y servidores al mismo tiempo, comunicándose directamente entre ellos y compartiendo recursos.

**¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?**
El middleware permite la comunicación y coordinación entre diferentes aplicaciones y equipos del sistema distribuido. Facilita funciones como comunicación, intercambio de datos y acceso a servicios. Se ubica entre el sistema operativo y las aplicaciones, formando una capa intermedia que oculta parte de la complejidad de la distribución.

**En la arquitectura de tres capas (3-tier), ¿Qué función cumple cada capa y por qué separarlas mejora el mantenimiento?**

Las tres capas son:

Presentación: muestra la información y recibe las acciones del usuario.

Lógica de negocio: procesa las reglas y operaciones de la aplicación.

Datos: almacena y administra la información.

Separarlas mejora el mantenimiento porque permite modificar una capa sin afectar directamente a las demás, facilitando también las pruebas, la actualización y la escalabilidad.

**¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?**
Los microservicios permiten dividir una aplicación en servicios independientes, facilitando el despliegue, escalabilidad y mantenimiento de cada componente. También permiten utilizar diferentes tecnologías según las necesidades. Como desafíos, introducen mayor complejidad de comunicación, monitoreo, seguridad, gestión de datos y despliegue.

**¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?**
La arquitectura peer-to-peer (P2P) es adecuada para BitTorrent porque los usuarios pueden compartir directamente partes de los archivos entre ellos. Esto distribuye la carga y evita depender completamente de un único servidor para transferir los archivos.

**¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?**
La escalabilidad vertical consiste en aumentar los recursos de un mismo servidor, como RAM, CPU o almacenamiento. La escalabilidad horizontal consiste en agregar más servidores o nodos. Las arquitecturas distribuidas favorecen especialmente la escalabilidad horizontal, ya que permiten distribuir la carga entre varios equipos.

**¿Qué significa transparencia en un sistema distribuido y menciona dos tipos de transparencia?**
La transparencia significa que el sistema oculta al usuario la complejidad de que los recursos y procesos estén distribuidos entre diferentes computadores. Dos tipos son:

Transparencia de ubicación: el usuario no necesita conocer dónde se encuentra un recurso.

Transparencia de acceso: permite utilizar un recurso sin preocuparse por la forma específica en que se accede a él.

**Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?**
Una arquitectura peer-to-peer puede continuar funcionando porque no depende de un único servidor central. Los demás nodos pueden seguir comunicándose y compartiendo recursos entre ellos, aunque algunos servicios podrían verse afectados dependiendo del diseño de la aplicación.

**En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?**
Los principales criterios fueron la cantidad de usuarios, necesidad de escalabilidad, disponibilidad, distribución de recursos, tolerancia a fallos, facilidad de mantenimiento y forma de comunicación entre los componentes. También se consideraron los requisitos específicos de la aplicación y la necesidad de distribuir la carga entre diferentes nodos.